

Object-Oriented Programming (OOP) Concepts

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

أحمد أكرم كامل عامر

15210436

1. Encapsulation:

- One of 4-pillars of oop.
- Mechanism of bundling data (fields) and the methods that operate on that data into single unit (Class), while restricting direct access to some of the object's components.
- Protects the internal state of an object from unintended or harmful modifications from outside code.

Core Access Modifiers (A: Accessible, NA: Not Accessible)

Modifier	Class	Package	Subclass	World
Private	A	NA	NA	NA
Default	A	A	NA	NA
Protected	A	A	A	NA
Public	A	A	A	A

Code Example:

```
// Encapsulation — BankAccount class
public class BankAccount {
    private String owner; // hidden from outside
    private double balance; // cannot be set directly

    public BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Getter — read-only access
    public double getBalance() {
        return balance;
    }

    // Controlled mutation with validation
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient funds!");
    }
}

// Usage
BankAccount acc = new BankAccount("Alice", 1000);
acc.deposit(500);
System.out.println(acc.getBalance()); // 1500.0
```

2. Inheritance:

- Allow a class (Sub class/Child) to acquire the properties (data) and behaviors (methods) of another class (Super class/Parent).
- This promotes code reuse.
- Establishes an is-a relationship, and supports hierarchical classification.
- In Java, inheritance is achieved with the extends keyword.

Types of Inheritance

Type	Description	Supported?
Single	One child inherits from one parent	Yes
Multilevel	A → B → C (chain of inheritance)	Yes
Hierarchical	Multiple children from one parent	Yes
Multiple	One child from multiple parents (via class)	No (Use Interfaces)

Code Example:

```
// Parent (Superclass)
public class Animal {
    private String name;

    public Animal(String name) { this.name = name; }

    public String getName() { return name; }

    public void eat() {
        System.out.println(name + " is eating.");
    }
}

// Child class — extends Animal
public class Dog extends Animal {
    private String breed;

    public Dog(String name, String breed) {
        super(name); // call parent constructor
        this.breed = breed;
    }

    public void bark() {
        System.out.println(getName() + " says: Woof!");
    }
}

// Multilevel: GuideDog extends Dog
public class GuideDog extends Dog {
    public GuideDog(String name) { super(name, "Labrador"); }

    public void guide() {
        System.out.println(getName() + " is guiding its owner.");
    }
}

// Usage
Dog d = new Dog("Rex", "Husky");
d.eat(); // inherited from Animal
d.bark(); // defined in Dog
```

3. Polymorphism:

- Poly = Many, Morphe = From -> Many Forms.
- Allows objects to take on multiple forms.
- A single method or interface can behave differently depending on the object that invokes it.

Types of Polymorphism

Type	When Resolved	Mechanism	Also Known as
Compile-Time Polymorphism	At Compile Time	Method Overloading	Static Dispatch
Run-Time Polymorphism	At Run Time	Method Overriding	Dynamic Dispatch

Code Example:

- Overloading (Compile-Time)

```
public class MathOps {  
    // Same name, different parameter types  
    public int add(int a, int b) { return a + b; }  
    public double add(double a, double b) { return a + b; }  
    public int add(int a, int b, int c) { return a + b + c; }  
}
```

```
MathOps m = new MathOps();  
m.add(2, 3);           // → 5 (int version)  
m.add(2.5, 3.1);       // → 5.6 (double version)  
m.add(1, 2, 3);        // → 6 (three-arg version)
```

- Overriding (Run-Time)

```
public class Animal {  
    public void makeSound() {  
        System.out.println("Some generic animal sound");  
    }  
}  
  
public class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Woof! Woof!");  
    }  
}  
  
public class Cat extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Meow!");  
    }  
}  
  
// Runtime polymorphism — decided at run-time  
Animal a1 = new Dog();  
Animal a2 = new Cat();  
a1.makeSound(); // Woof! Woof!  
a2.makeSound(); // Meow!
```

4. **Abstraction:**

- Exposing only the relevant details of an object while hiding the complex internal implementation.
- It lets programmers work with high-level concepts without needing to understand underlying complexity.

Abstraction Mechanisms

Feature	Abstract Class	Interface
Keyword	abstract class	Interface
Has constructor?	Yes	No
Has Fields ?	Yes(Any Type)	Yes(only public static final)
Method Types	Abstract + Concrete	Abstract + default + static
Inheritance	extends (single)	implements (multiple)
Use When	Shared base behavior	Define a contract / capability

Code Example:

- Abstract Class

```
// Abstract class — cannot be instantiated
public abstract class Shape {
    private String color;

    public Shape(String color) { this.color = color; }

    // Abstract method — must be implemented by subclass
    public abstract double area();

    // Concrete method — shared by all shapes
    public void displayColor() {
        System.out.println("Color: " + color);
    }
}

public class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) {
        super(color); this.radius = radius;
    }
    @Override
    public double area() { return Math.PI * radius * radius; }
}

public class Rectangle extends Shape {
    private double width, height;
    public Rectangle(String color, double w, double h) {
        super(color); this.width = w; this.height = h;
    }
    @Override
    public double area() { return width * height; }
}
```

- Interface

```
// Interface — defines a contract
public interface Payment {
    void processPayment(double amount); // abstract
    default void printReceipt() {        // default
        System.out.println("Payment processed.");
    }
}

public class CreditCard implements Payment {
    @Override
    public void processPayment(double amount) {
        System.out.println("Charging $" + amount + " to credit card.");
    }
}

public class PayPal implements Payment {
    @Override
    public void processPayment(double amount) {
        System.out.println("Sending $" + amount + " via PayPal.");
    }
}

// Usage
Payment p1 = new CreditCard();
Payment p2 = new PayPal();
p1.processPayment(99.99);
p2.processPayment(49.99);
```

Summary:

Concept	Core Idea	Keywords	Benefit
Encapsulation	Hide data, expose via methods	private, public, getter	Data safety & control
Inheritance	Child reuses parent behavior	extends, super	Code reuse, hierarchy
Polymorphism	One name, many behaviors	@Override, overload	Flexibility & extensibility
Abstraction	Show what, hide how	abstract, interface	Simplicity & modularity